

Practical :1

1. Introduction to Security Tools and Environment Setup: Configure basic cybersecurity tools and environment for performing security-related tasks.

Aim

To understand and configure a basic cybersecurity environment using online tools and perform security-related tasks such as system information gathering, network connectivity testing, DNS resolution, port verification, and security environment validation.

Objective

1. To understand the role of cybersecurity tools in information security.
2. To configure a basic cybersecurity environment using an online platform.
3. To perform system and network information gathering.
4. To understand DNS resolution and connectivity testing.
5. To verify network services through port checking.
6. To gain practical exposure to cybersecurity operations using Python.

Tool Used

Google Colab (Online Platform)

Google Colab is a cloud-based Jupyter Notebook environment that allows users to write and execute Python programs directly from a web browser without installing any software.

Advantages of Google Colab

- No software installation required.
- Free cloud-based platform.
- Supports Python programming.
- Accessible from any device with internet access.
- Suitable for cybersecurity learning and experimentation.

What is a Cybersecurity Environment?

A cybersecurity environment is a controlled setup containing tools, configurations, and resources required to perform security-related tasks such as:

- Security monitoring
- Network analysis
- Vulnerability assessment
- Ethical hacking
- Incident response
- Digital forensics

Before performing security operations, cybersecurity professionals must verify:

- System configuration
- Network connectivity
- DNS functionality
- Port accessibility
- Secure communication channels

Cybersecurity Concepts Covered

1. Hostname

A hostname is the unique name assigned to a computer on a network.

Example:

DESKTOP-12345

2. IP Address

An IP address uniquely identifies a device connected to a network.

Example:

192.168.1.10

3. DNS Resolution

DNS (Domain Name System) converts domain names into IP addresses.

Example:

www.google.com → 142.250.193.196

4. Port

A port is a logical communication endpoint used by network services.

Port	Service
80	HTTP
443	HTTPS
21	FTP
22	SSH

5. HTTPS

HTTPS (HyperText Transfer Protocol Secure) ensures secure communication through encryption.

Software Requirement

- Google Account
- Internet Connection
- Web Browser
- Google Colab

Practical Procedure

Step 1

Open Google Colab:

<https://colab.research.google.com>

Step 2

Create a New Notebook.

Step 3

IS_PRACTICAL_1

Program

```
# =====
# PRACTICAL NO. 1
# INTRODUCTION TO SECURITY TOOLS AND ENVIRONMENT SETUP
# =====

# Import required libraries

import socket
import platform
import requests
from datetime import datetime

# =====
# PRACTICAL HEADER
```

```

# =====

print("=" * 70)
print("CYBERSECURITY ENVIRONMENT SETUP USING GOOGLE COLAB")
print("=" * 70)

# =====
# DATE AND TIME INFORMATION
# =====

print("\nCURRENT DATE AND TIME")
print("-" * 70)

current_time = datetime.now()
print("Current Date and Time :", current_time)

# =====
# SYSTEM INFORMATION
# =====

print("\nSYSTEM INFORMATION")
print("-" * 70)

print("Operating System :", platform.system())
print("System Release   :", platform.release())
print("System Version    :", platform.version())
print("Machine Type      :", platform.machine())
print("Processor         :", platform.processor())

# =====
# HOST INFORMATION
# =====

print("\nHOST INFORMATION")
print("-" * 70)

hostname = socket.gethostname()

print("Hostname :", hostname)

try:
    ip_address = socket.gethostbyname(hostname)
    print("IP Address :", ip_address)
except:

```

```

    print("Unable to Retrieve IP Address")

# =====
# INTERNET CONNECTIVITY TEST
# =====

print("\nINTERNET CONNECTIVITY TEST")
print("-" * 70)

try:
    response = requests.get("https://www.google.com")

    print("Connection Status : SUCCESS")
    print("HTTP Status Code :", response.status_code)

except:
    print("Connection Status : FAILED")

# =====
# DNS RESOLUTION TEST
# =====

print("\nDNS RESOLUTION TEST")
print("-" * 70)

try:
    google_ip = socket.gethostbyname("www.google.com")

    print("Domain Name : www.google.com")
    print("Resolved IP :", google_ip)

except:
    print("DNS Resolution Failed")

# =====
# PORT CONNECTIVITY TEST
# =====

print("\nPORT CONNECTIVITY TEST")
print("-" * 70)

host = "google.com"

ports = [80, 443]

```

```

for port in ports:

    try:
        connection = socket.create_connection((host, port), timeout=5)

        print(f"Port {port} : OPEN")

        connection.close()

    except:
        print(f"Port {port} : CLOSED")

# =====
# WEBSITE SECURITY CHECK
# =====

print("\nWEBSITE SECURITY CHECK")
print("-" * 70)

url = "https://www.google.com"

if url.startswith("https://"):
    print("Secure HTTPS Protocol Detected")
else:
    print("Insecure Protocol")

# =====
# CYBERSECURITY ENVIRONMENT STATUS
# =====

print("\nCYBERSECURITY ENVIRONMENT STATUS")
print("-" * 70)

print("✓ Python Environment Ready")
print("✓ Internet Connectivity Verified")
print("✓ DNS Resolution Working")
print("✓ Port Connectivity Verified")
print("✓ HTTPS Security Verified")
print("✓ Security Environment Ready")

print("\nPractical Completed Successfully")

print("=" * 70)

```

Detailed Step-by-Step Explanation of the Program

Program Overview

This program demonstrates how to configure a basic cybersecurity environment using Google Colab. It performs the following tasks:

1. Collects system information.
2. Retrieves hostname and IP address.
3. Tests internet connectivity.
4. Performs DNS resolution.
5. Checks network ports.
6. Verifies secure HTTPS communication.
7. Displays the final cybersecurity environment status.

Step 1: Import Required Libraries

```
import socket
import platform
import requests
from datetime import datetime
```

Explanation

Python libraries provide ready-made functions.

A. socket Library

```
import socket
```

Purpose:

- Network communication.
- IP address retrieval.
- DNS lookup.
- Port connectivity testing.

Examples:

```
socket.gethostname()
socket.gethostbyname()
socket.create_connection()
```

Used extensively in cybersecurity and network analysis.

B. platform Library

```
import platform
```

Purpose:

Collect operating system information.

Examples:

```
platform.system()
platform.release()
platform.version()
```

Used by security analysts to identify system details.

C. requests Library

```
import requests
```

Purpose:

Send HTTP/HTTPS requests to websites.

Example:

```
requests.get("https://www.google.com")
```

Used for:

- Website testing
- API communication
- Security validation

D. datetime Library

```
from datetime import datetime
```

Purpose:

Retrieve current date and time.

Example:

```
datetime.now()
```

Used for:

- Logging
- Auditing
- Incident tracking

Step 2: Display Practical Heading

```
print("=" * 70)
print("CYBERSECURITY ENVIRONMENT SETUP USING GOOGLE COLAB")
print("=" * 70)
```

Explanation

Line 1

```
"=" * 70
```

Creates 70 equal signs.

Output:

```
=====
```

Purpose:

- Makes output organized.
- Separates sections visually.

Line 2

```
print("CYBERSECURITY ENVIRONMENT SETUP USING GOOGLE COLAB")
```

Displays the title.

Output:

```
CYBERSECURITY ENVIRONMENT SETUP USING GOOGLE COLAB
```

Step 3: Display Current Date and Time

```
current_time = datetime.now()
print("Current Date and Time :", current_time)
```

Explanation

```
datetime.now()
```

```
datetime.now()
```

Retrieves current system date and time.

Example Output:

2026-06-12 14:35:45

Variable Storage

```
current_time = datetime.now()
```

Stores the value in the variable.

Example:

```
current_time = 2026-06-12 14:35:45
```

Print Statement

```
print("Current Date and Time :", current_time)
```

Output:

Current Date and Time : 2026-06-12 14:35:45

Cybersecurity Importance

Every security event requires timestamps.

Examples:

- Login logs
- Attack logs
- Audit records
- Incident reports

Step 4: Collect System Information

```
print("Operating System :", platform.system())
```

Explanation

```
platform.system()
```

Returns OS name.

Example:

Linux

or

Windows

Output

Operating System : Linux

OS Release

```
platform.release()
```

Example Output:

5.15.109+

Purpose:

Identifies operating system release.

OS Version

```
platform.version()
```

Provides detailed version information.

Example:

#1 SMP Tue May 30

Machine Type
`platform.machine()`

Output:
x86_64

Meaning:
64-bit architecture.
Processor Information
`platform.processor()`

Example:
Intel Core i5

or
x86_64

Why Collect System Information?
Cybersecurity professionals identify:

- OS vulnerabilities
- System compatibility
- Attack surfaces

Example:
Older operating systems may contain known vulnerabilities.

Step 5: Host Information
`hostname = socket.gethostname()`

Explanation
Every device on a network has a hostname.

Example:
DESKTOP-PC

or
localhost

Display Hostname
`print("Hostname :", hostname)`

Output:
Hostname : localhost

Why Important?
Hostnames help identify systems within a network.

Used in:

- Network management
- Security monitoring
- Device tracking

Step 6: Retrieve IP Address
`ip_address = socket.gethostbyname(hostname)`

Explanation

Converts hostname into IP address.

Example:

localhost

becomes

127.0.0.1

Output

IP Address : 127.0.0.1

Why Important?

IP addresses uniquely identify devices.

Cybersecurity uses IP addresses for:

- Tracking attackers
- Monitoring systems
- Network analysis

Step 7: Exception Handling

try:

Means:

Attempt execution.

except:

Means:

If error occurs, handle it safely.

Example:

try:

```
ip_address = socket.gethostbyname(hostname)
```

except:

```
print("Unable to Retrieve IP Address")
```

Why Important?

Prevents application crashes.

Professional security tools must continue operating even when errors occur.

Step 8: Internet Connectivity Test

```
response = requests.get("https://www.google.com")
```

What Happens?

Step 1

Program sends request.

Program

↓

Google Server

Step 2

Google responds.

Google Server

↓

Response

Response Object

Stored in:
response

Contains:

- Status code
- Headers
- Content

Status Code

response.status_code

Common Status Codes

Code	Meaning
200	Success
301	Redirect
403	Forbidden
404	Not Found
500	Server Error

Output

Connection Status : SUCCESS

HTTP Status Code : 200

Cybersecurity Importance

Verifies:

- Internet availability
- Server accessibility
- Website response

Step 9: DNS Resolution

```
google_ip = socket.gethostbyname("www.google.com")
```

Explanation

Converts domain name into IP address.

Example:

www.google.com

becomes

142.250.193.196

Output

Google IP Address : 142.250.193.196

Why DNS Matters?

Humans remember:

google.com

Computers use:

142.250.193.196

DNS performs this translation.

Step 10: Port Connectivity Test

host = "google.com"

Target website.

ports = [80,443]

Ports to test.

Port 80

HTTP

Port 443

HTTPS

Loop

for port in ports:

Runs twice:

1. Port 80
2. Port 443

Create Connection

socket.create_connection((host,port),timeout=5)

Attempts connection.

Example:

google.com:80

Possible Results

Open

Port 80 : OPEN

Closed

Port 80 : CLOSED

Why Important?

Cybersecurity professionals use port analysis to identify:

- Running services
- Security weaknesses
- Open attack vectors

Step 11: HTTPS Security Check

url="https://www.google.com"

Website URL.

url.startswith("https://")

Checks whether URL begins with HTTPS.

HTTPS Meaning

HyperText Transfer Protocol Secure

Provides:

- Encryption

- Confidentiality
- Data Integrity

Output

Secure HTTPS Protocol Detected

Why Important?

HTTPS protects:

- Passwords
- Banking information
- Personal data

from interception.

Step 12: Cybersecurity Environment Status

print("✓ Python Environment Ready")

Displays successful completion.

Output

- ✓ Python Environment Ready
- ✓ System Information Collected
- ✓ Host Information Retrieved
- ✓ Internet Connectivity Verified
- ✓ DNS Resolution Working
- ✓ Port Verification Completed
- ✓ Secure HTTPS Communication Verified
- ✓ Cybersecurity Environment Successfully Configured

Cybersecurity Skills Learned

Skill	Purpose
System Enumeration	Collect OS and hardware information
Host Discovery	Identify hostname and IP
Network Testing	Verify internet access
DNS Lookup	Resolve domain names
Port Verification	Check active services
HTTPS Validation	Verify secure communication
Exception Handling	Manage runtime errors
Python Security Automation	Automate cybersecurity tasks

This level of explanation is suitable for a practical manual, lab record, viva preparation, and university submission.

Expected Output

The program will display:

- Current Date and Time
- Operating System Information
- Machine Information

- Hostname and IP Address
- Internet Connectivity Status
- DNS Resolution Result
- Open Port Information
- HTTPS Security Status

CYBERSECURITY ENVIRONMENT SETUP USING GOOGLE COLAB

CURRENT DATE AND TIME

Current Date and Time : 2026-06-12 04:53:40.267564

SYSTEM INFORMATION

Operating System : Linux
OS Release : 6.6.122+
OS Version : #1 SMP Thu Apr 30 18:17:14 UTC 2026
Machine Type : x86_64
Processor : x86_64

HOST INFORMATION

Hostname : f497ff0076a5
IP Address : 172.28.0.12

INTERNET CONNECTIVITY TEST

Connection Status : SUCCESS
HTTP Status Code : 200

DNS RESOLUTION TEST

Domain Name : www.google.com
Resolved IP : 142.251.152.119

PORT CONNECTIVITY TEST

Port 80 : OPEN
Port 443 : OPEN

SECURITY PROTOCOL CHECK

Secure HTTPS Protocol Detected

CYBERSECURITY ENVIRONMENT STATUS

- ✓ Python Environment Ready
- ✓ System Information Collected
- ✓ Host Information Retrieved

- ✓ Internet Connectivity Verified
- ✓ DNS Resolution Working
- ✓ Port Verification Completed
- ✓ Secure HTTPS Communication Verified
- ✓ Cybersecurity Environment Successfully Configured

Practical Completed Successfully

Result

The cybersecurity environment was successfully configured using Google Colab. System information, network connectivity, DNS resolution, port verification, and HTTPS security validation were performed successfully.